```
class AddCustomerNameIndexToOrders < ActiveRecord::Migration
  def change
    add_index :orders, :name
  end
end
```

If we give `add_index` the optional parameter `unique: true`, a unique index will be created, forcing values in the indexed column to be unique.

By default the index will be given the name *index_table_on_column*. We can override this using the `name: "somename"` option. If we use the `:name` option when adding an index, we'll also need to specify it when removing the index.

We can create a *composite index*—an index on multiple columns—by passing an array of column names to `add_index`.

Indices are removed using the `remove_index` method.

Primary Keys

Rails assumes every table has a numeric primary key (normally called `id`) and ensures the value of this column is unique for each new row added to a table. We'll rephrase that.

Rails doesn't work too well unless each table has a primary key that Rails can manage. By default, Rails will create numeric primary keys, but you can also use other types such as UUIDs, depending on what your actual database provides. Rails is less fussy about the name of the column. So for your average Rails application, our strong advice is to go with the flow and let Rails have its `id` column.

If you decide to be adventurous, you can start by using a different name for the primary key column (but keeping it as an incrementing integer). Do this by specifying a `:primary_key` option on the `create_table` call:

```
create_table :tickets, primary_key: :number do |t|
  t.text :description
```